

Distributed Training

The Rise of Big AI

- AI models are becoming extremely large very quickly.
 - Bigger models can understand more complex patterns in data.
 - Training these huge models needs massive computing power.
 - Normal systems are no longer enough for modern AI workloads.
-

Computational Limits

- Training large AI models requires huge computational resources.
 - A single processor or machine has limitations.
 - Training on one machine can take days, weeks, or even months.
 - This becomes a bottleneck for AI innovation and scaling.
-

Parallelism

- Parallelism means using multiple processors/devices together.
 - The workload is divided across different machines or GPUs.
 - Helps reduce training time significantly.
 - Makes it possible to train much larger models efficiently.
-

Data Parallelism

Basic Idea

- Training data is split into smaller chunks/subsets.
- Multiple processors train the same model simultaneously.
- Each processor works on a different portion of data.
- After every training step, processors synchronize updates.

Important Point

- Useful when:
 - The model fits inside one processor/GPU memory.

- But the dataset is too large for fast training on one machine.

Simple Understanding

- Same model copy everywhere.
 - Different data on each processor.
-

Model Parallelism

Basic Idea

- The model itself is divided across multiple processors.
- Each processor handles only a part of the model.
- Needed when the model is too large for a single GPU.

Key Notes

- More difficult to implement compared to data parallelism.
- Requires communication between processors frequently.

Simple Understanding

- Different parts of the model on different devices.
-

Hybrid Parallelism

- Combination of data parallelism + model parallelism.
 - Example:
 - Data parallelism across multiple machines.
 - Model parallelism within each machine.
 - Commonly used for very large AI systems.
-

Challenges in Distributed Training

Major Challenges

- Increased software and hardware complexity.
- Communication overhead between processors.
- Synchronization becomes difficult.

- Proper load balancing is important.
 - Every processor should get equal work.

Bottlenecks

- Too much communication can slow training.
 - Coordination across machines is expensive.
-

Ray

What is Ray?

- Open-source distributed computing framework.
- Mainly used with Python.
- Developed for scalable distributed applications.

Main Purpose

- Run Python code efficiently across clusters/machines.
 - Simplifies distributed computing.
-

Ray Tasks

- Ray allows parallel execution using simple decorators.
 - `@ray.remote` is used to create distributed tasks.
 - Tasks help parallelize Python functions easily.
-

Ray Actors

- Actors are stateful distributed objects.
 - Useful when state/data must persist across operations.
 - Helps build scalable distributed systems.
-

Anyscale

About

- Built on top of Ray.

- Provides a production-ready AI platform.

Purpose

- Simplifies:
 - Development
 - Deployment
 - Management of AI workloads

Key Feature

- Unified platform for the entire AI lifecycle.
-

Key Benefits of Ray & Anyscale

- Fully managed platform support.
 - Easy scalability from laptop to cluster.
 - Faster development and experimentation.
 - Cost-efficient distributed computing.
-

Unleashing the Power of Ray

- Enables scalable distributed computing in Python.
 - Great for:
 - Machine learning
 - Data processing
 - AI workloads
-

Why Use Ray?

- Scale code effortlessly from local machine to cluster.
- Supports:
 - CPUs
 - GPUs
 - Multi-node systems

- Excellent for ML pipelines and deployment systems.
-

Core Components of Ray

Ray Core

- Handles parallel and distributed execution.

Ray Datasets

- Used for scalable data processing and handling.

Ray Train

- Supports distributed model training.

Ray Tune

- Used for hyperparameter optimization.
-

Ray Core in Action

Key Concepts

- Use `@ray.remote` for parallel tasks.
- `ray.get()` retrieves results.
- `ray.put()` stores objects in distributed memory.

Advantage

- Works both locally and on clusters seamlessly.
-

Ray Tune

Purpose

- Runs hyperparameter tuning/search in parallel.

Integration

- Works with:
 - PyTorch
 - TensorFlow
 - XGBoost

Additional Support

- Integrates with:
 - Optuna
 - HyperOpt
-

Ray Train & Ray Serve

Ray Train

- Helps scale model training with minimal code changes.

Ray Serve

- Used for production deployment of ML models.
 - Supports:
 - Autoscaling
 - Traffic routing
-

Companies Using Ray

Examples

- OpenAI → scalable reinforcement learning
 - Netflix → recommendation systems
 - Uber → deep learning infrastructure
 - Shopify → demand forecasting
-

Final Summary

- Large AI models require distributed training.
- Parallelism solves computational bottlenecks.
- Data parallelism → split data.
- Model parallelism → split model.
- Hybrid parallelism → combine both approaches.
- Ray simplifies distributed computing in Python.

- Anyscale provides enterprise-level management for Ray workloads.